PNI ソフトウェア健全性診断レポート

本レポートは、WordPress 会員分析プラグイン `Member Analytics Portal` (`wp_plugin_messy` **v1.3.0)** に対して、PNI ステートフル監査ワークフロー `php_wp_plugin_messy`（3観点アーキテクチャ記録・PHP/WordPress AST 静的解析・隔離 Docker コンテナによる PHP 8.2 構文および WordPress フック登録スモーク検証・Trivy 脆弱性スキャン）を実行した包括的診断結果である。



Member Analytics Portal (v1.3.0) PNI 総合健全性スコアカード

診断メタデータ概要

- **対象プロジェクト**: `Member Analytics Portal (wp_plugin_messy)` (`v1.3.0` / 対象ルート: `examples/php/wp_plugin_messy`)
- **対象言語・ビルド基盤**: `php`（プラグインヘッダ指定 `Requires PHP: 7.4`、`Requires at least: 5.8`、`composer.json` 不在の `raw_php` 構成）
- **検出フレームワーク**: `wordpress`（WP REST API v1、Admin-AJAX、Shortcode API、Options API、`$wpdb`）
- **診断ワークフロー / Run ID**: `php_wp_plugin_messy` / `wf-31a8fefa`
- **評価実施日時**: `2026-10-24T09:20:09.804814+00:00`
- **総合スコア・グレード**: **20 / 100 (Grade D)**
- **エンジニアリングルール適合率**: `0 / 17` ルール PASS（`FAIL`: 14件、`WARN`: 2件、`NOT_APPLICABLE`: 1件）

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する改善ロードマップ

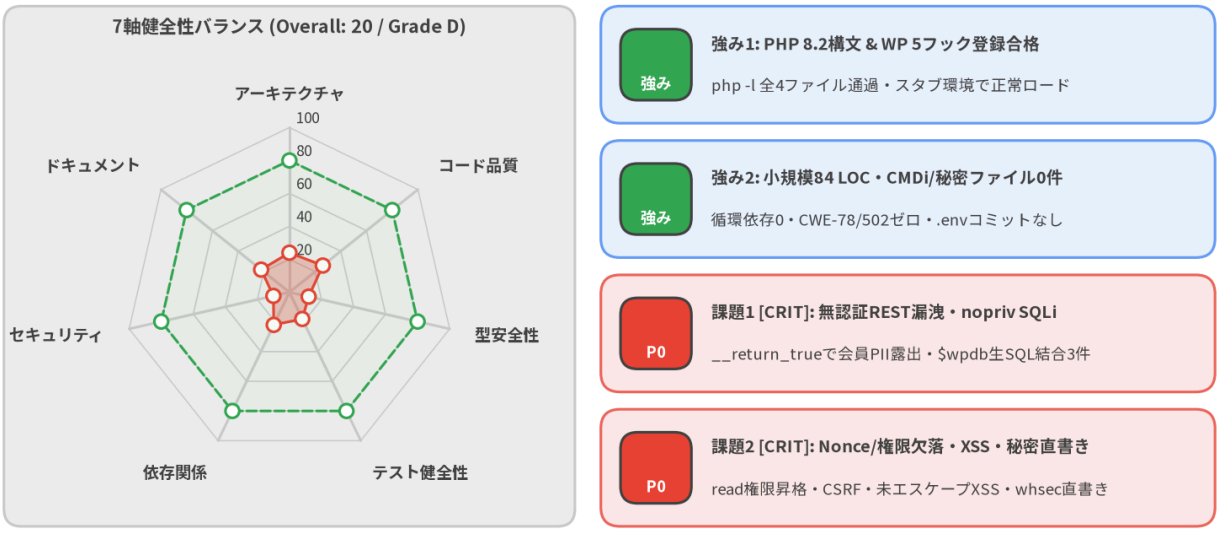
4. 評価指標とルール適合マトリクス

5. コード品質・型安全性・ランタイム検証詳細

6. セキュリティ・脆弱性・サプライチェーン分析

1. エグゼクティブサマリー

WordPress プラグイン `Member Analytics Portal` (`examples/php/wp_plugin_messy` , v1.3.0) の総合健全性スコアは **20/100 (Grade D)** と診断された。PHP 8.2 隔離コンテナでの `php -l` 構文検証と5フック登録スモークテスト自体は通過するものの、REST API (`/wp-json/member-analytics/v1/export`) の `permission_callback => '__return_true'` による会員 PII 無認証露出、`wp_ajax_nopriv_map_search_members` の未認証 SQL インジェクション、`wp_ajax_map_purge_logs` および設定画面（`capability => 'read'`）の Nonce/権限検証欠落、3箇所の `$wpdb` 生 SQL 文字列結合（`CWE-89`）、4箇所の未エスケープ XSS（`CWE-79`）、Webhook シークレット直書き、全4ファイルの `ABSPATH` ガード不在など、即時悪用可能な致命的脆弱性が集中している。

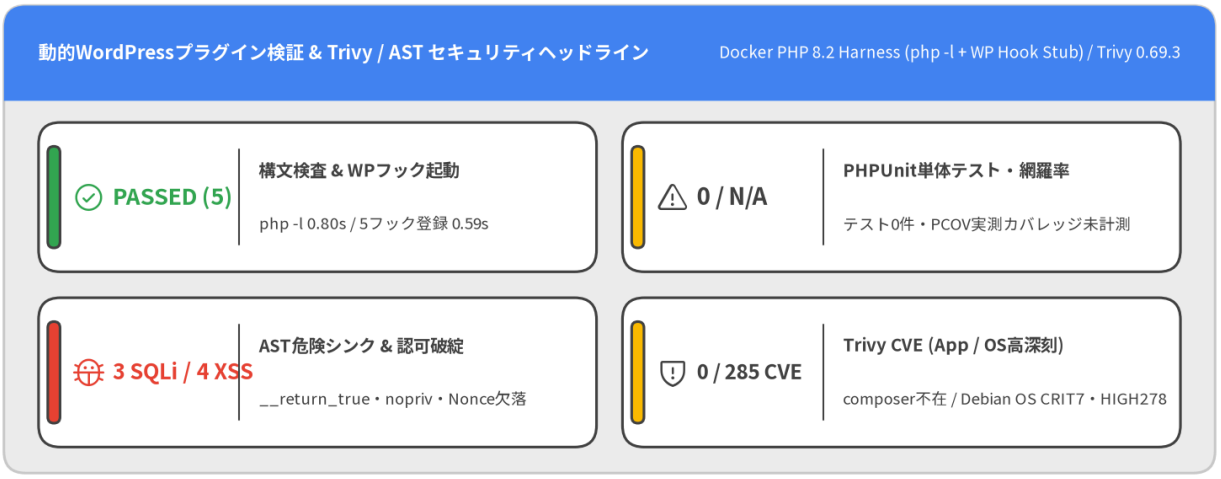


7軸エンジニアリング健全性レーダーと強み・最優先リスク (P0) 要約

7軸評価サマリーと主要診断結論

- **全7軸が基準値（80点）を大幅に下回る緊急リスク集中プロファイル**: セキュリティ（`10`）・型安全性（`12`）・テスト健全性（`18`）が危機的水準にあり、依存関係（`22`）・ドキュメント（`22`）・アーキテクチャ（`24`）・コード品質（`26`）を含め、WordPress プラグイン審査基準（Plugin Check / OWASP ASVS）に対して抜本的な改修を要する。

- **最低スコア評価ステップ（昇順）**：`php_test_audit`（`5/100`：自動テスト0件）、`security_vulnerability_audit`（`8/100`：無認証 REST/AJAX SQLi・XSS・秘密鍵直書き）、`php_ast_audit`（`14/100`：`strict_types=1` 0%・WordPress 規約違反16件）が総合スコアを大きく押し下げている。



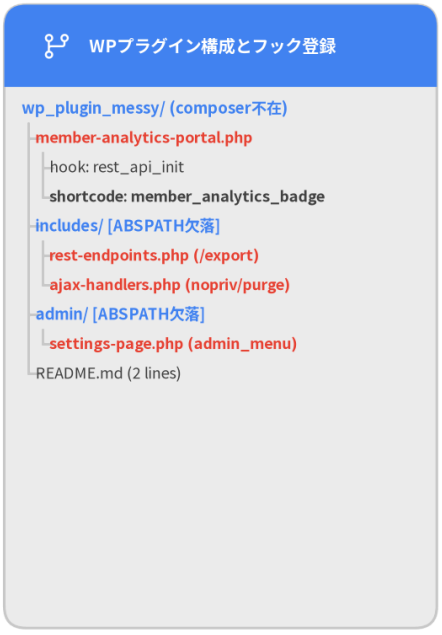
動的プラグイン検証 & Trivy セキュリティ監査ヘッドライン指標

エグゼクティブ結論

- **強み (Verified Strengths)**: 標準プラグインヘッダを備え、PHP 8.2 コンテナ上での `php -l` 構文検査（`0.80s`）と WordPress フック登録スモークテスト（`0.59s`、5フック登録）が通過している。またコード全体が `4` ファイル・`84` LOC と小規模で、OS コマンド実行（`CWE-78`）・危険な `unserialize`（`CWE-502`）・`.env`/秘密鍵ファイルの誤コミットも `0` 件であるため、短期間で完全な改修が可能である。
- **最優先アクション (P0 Priorities)**: 本番 WordPress サイトへの有効化前に、(1) 全4ファイルへの `ABSPATH` ガード追加、(2) REST `/export` の `current_user_can('list_users')` 化・`wp_ajax_nopriv_map_search_members` 廃止・設定画面の `'manage_options'` 化と `check_ajax_referer` / `check_admin_referer` 強制、(3) 全3クエリの `$wpdb->prepare()` パラメータ化と全出力の `esc_html()` / `esc_attr()` 遅延エスケープ、(4) `includes/rest-endpoints.php:3` のハードコード Webhook シークレット失効・外部化を即時断行すべきである。

2. アーキテクチャとソースコード構成

`Member Analytics Portal`（`wp_plugin_messy`）は `member-analytics-portal.php` **を起点に3つのサブファイルを無条件** `require_once` **読み込みする手続き型フック直結モノリス構成（計4ファイル・84 LOC・0クラス・7グローバル関数）** を採っている。PSR-4 名前空間や Service/Repository 層分離は存在せず、全4ファイルで直接 HTTP アクセスを遮断する `if (!defined('ABSPATH')) { exit; }` ガードが欠落している。

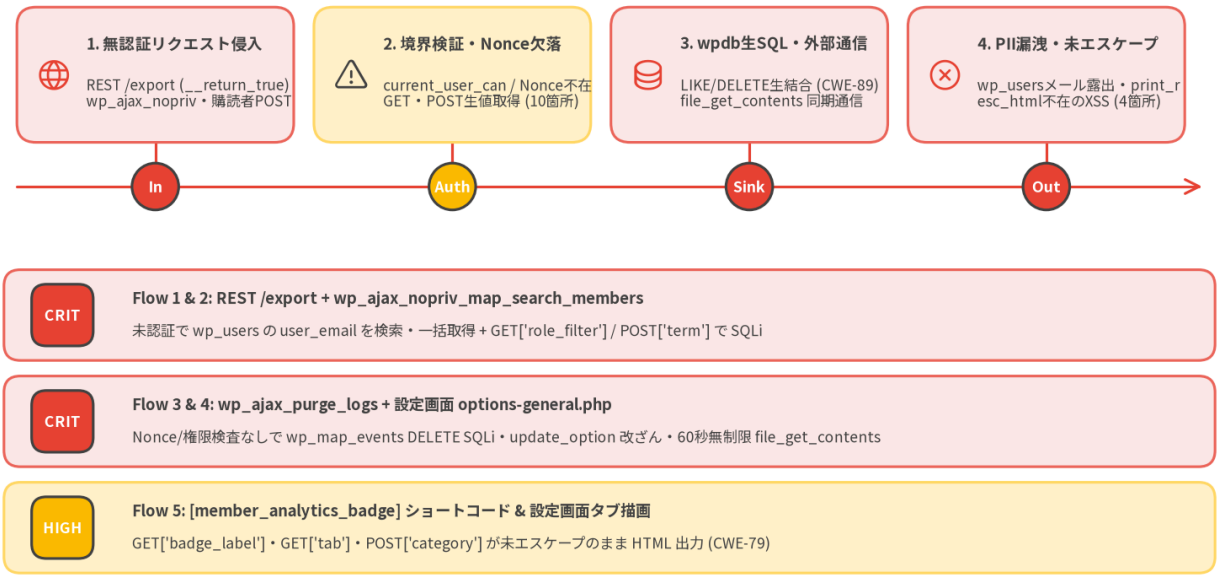


モジュールパス	LOC	公開関数・登録フックと主要責務・健全性メモ
admin/ settings-page.php	31	map_register_admin_menu, map_render_settings_page [Risk 1.00] ABSPATH欠落・read権限・Nonce不在・XSS・外部通信
includes/ ajax-handlers.php	21	map_ajax_search_members, map_ajax_purge_logs [Risk 1.00] ABSPATH欠落・nopriv未認証SQLi・DELETE SQLi・XSS
includes/ rest-endpoints.php	21	map_register_rest_routes, map_export_all_members [Risk 1.00] ABSPATH欠落・whsec直書き・__return_true・SQLi
member-analytics-portal.php	11	map_render_badge_shortcode | Requires PHP: 7.4 (EOL) [Risk 1.00] ABSPATH欠落・手動require_once 3件・バッジXSS

観点1: WordPressフック・サブファイル構成ツリーとABSPATH欠落モジュール構成表 (全4ファイル・84 LOC)

REST（`__return_true`）・Admin-AJAX（`nopriv`）から `$wpdb` 生 SQL 結合へ至るデータフロー

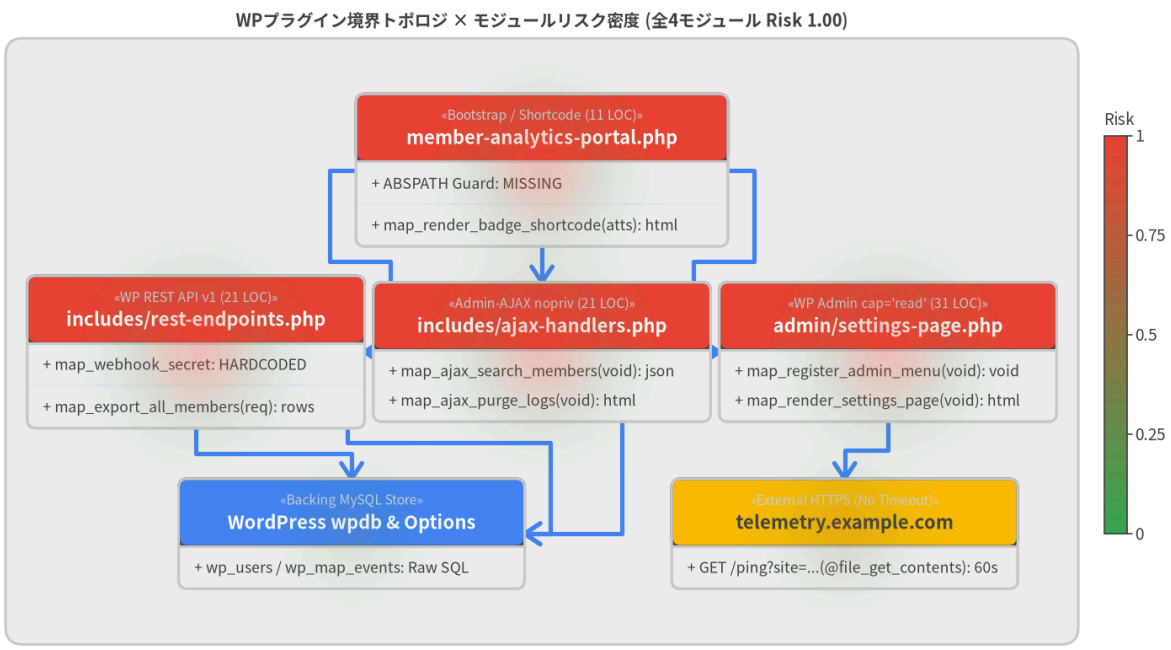
実行フロー解析（`ExecutionFlowMap`）により、外部リクエストから WordPress DB（`{$wpdb->users}` / `wp_map_events` / `wp_options`）および HTML レスポンスに至る5本の経路すべてでサニタイズ・認可・プリペアドステートメントが欠落していることが判明した。特に `/wp-json/member-analytics/v1/export` と `wp_ajax_nopriv_map_search_members` は未認証の第三者から直接 `$wpdb->get_results()` の文字列結合クエリへ到達する。



観点2: REST (__return_true) & wp_ajax_nopriv から wpdb 生SQL結合・XSS出力へ至る危険データフロー

WordPress プラグイン実行・検証境界トポロジとリスク密度

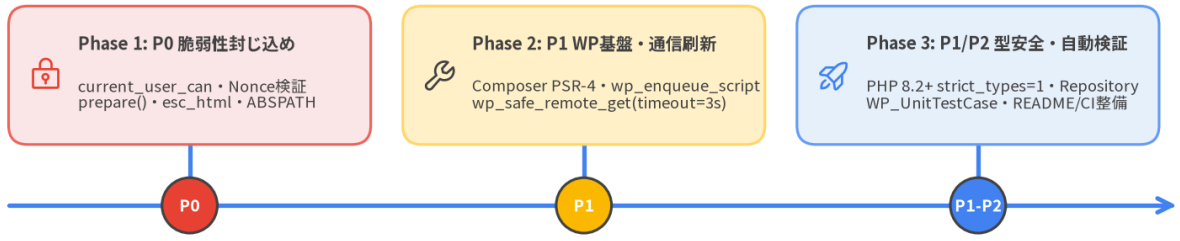
指摘密度から算出された `module_hotspots` では、**4つのプラグイン `.php` ファイルすべてが `risk_score=1.00`**（`includes/rest-endpoints.php`: CRITICAL 3 / MED 2、`includes/ajax-handlers.php`: CRITICAL 3、`admin/settings-page.php`: CRITICAL 1 / HIGH 2 / MED 1、`member-analytics-portal.php`: HIGH 5 / MED 1 / LOW 1）に達しており、全モジュールがグローバルな `$wpdb` とスーパーグローバル配列に密結合している。



観点3: WPプラグイン実行・データストア境界トポロジ × リスク密度オーバーレイ (HeatmapOverlay)

3. 推奨する改善ロードマップ

検出された全21件の指摘事項（`CRITICAL` 7件、`HIGH` 7件、`MEDIUM` 6件、`LOW` 1件）を解消し、総合健全性スコアを Grade D (`20`) から本番 WordPress サイトへ安全に導入できる Grade A 水準（ `85+` ）へ引き上げるため、以下の **Phase 1（P0 緊急止血：認可・CSRF・SQLi・XSS・秘密鍵・ABSPATH 封じ込め） -> Phase 2（P1 構造刷新：Composer/PSR-4・`wp_enqueue_script`・`wp_safe_remote_get` タイムアウト化） -> Phase 3（P1/P2 品質保証：PHP 8.2+ `strict_types=1`・PHPUnit/`WP_UnitTestCase` 自動テスト・ガバナンス整備）** の3フェーズ改善計画を推奨する。



3フェーズ推奨改善ロードマップ (Phase 1 P0封じ込め -> Phase 2 基盤近代化 -> Phase 3 型・自動テスト)

フェーズ別実行計画とトレーサビリティ

- **Phase 1 (`P0` ― 即時止血：認可・CSRF・SQLi・XSS・シークレット漏洩の完全封じ込め)**: 全4ファイル先頭に `if (!defined('ABSPATH')) { exit; }` を配置し、`includes/rest-endpoints.php` の `'__return_true'` を `current_user_can('list_users')` へ変更、`wp_ajax_nopriv_map_search_members` を廃止、`map_ajax_purge_logs` と `settings-page.php`（`capability => 'manage_options'`）に `check_ajax_referer()` / `check_admin_referer()` を導入する。同時に全3箇所の `$wpdb` クエリを `$wpdb->prepare()` + `{$wpdb->prefix}map_events` へ

書き換え、全4箇所の HTML/属性出力に `esc_html()` / `esc_attr()` を適用し、直書き `$map_webhook_secret` を失効・外部注入化する。

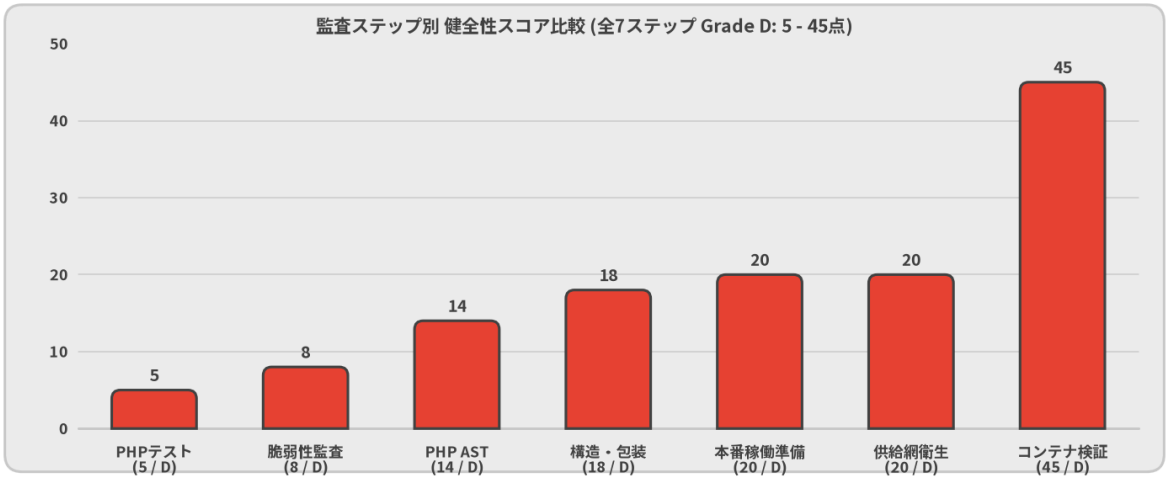
- **Phase 2 (`P1` ー 短期優先：Composer/PSR-4・WordPress 標準 API 移行・ワーカー枯渇防止)**: `composer.json` （ `php: ^8.2` 、PSR-4 オートローディング）と `composer.lock` を作成し、 `Requires PHP: 8.2` へ引き上げる。管理画面の直書き `<script>` タグを `admin_enqueue_scripts` + `wp_enqueue_script()` に移行し、 `@file_get_contents()` を `timeout => 3` 秒・ `is_wp_error()` 検査付きの `wp_safe_remote_get()` （または WP-Cron 非同期ジョブ）へ置換、 `print_r()` デバッグ出力を除去して PSR-3 構造化ログへ統一する。
- **Phase 3 (`P1/P2` ー 中期定着：厳格型宣言・PHPUnit/ `WP_UnitTestCase` 自動テスト・ガバナンス確立)**: 全ファイルに `declare(strict_types=1);` ・ネイティブ型宣言・PHPDoc を整備して `$wpdb` を Repository クラスへ分離し、PHPUnit 11 + `WP_UnitTestCase` / `WP_Ajax_UnitTestCase` + PCOV によりオフライン（ `--network=none` ）で 80% 以上のカバレッジを計測する。併せて `README.md` ・ `LICENSE` ・ `SECURITY.md` ・ `.gitignore` ・ GitHub Actions CI を整備する。

優先度	改善施策タイトル	対象ファイル	期待効果 (Expected Impact)	関連ルール / CWE
P0	$wpdb->prepare 全SQL化と esc_html/esc_attr 遅延エスケープ	全4 PHPファイル	CWE-89 SQLi (3件) と CWE-79 XSS (全経路) を根絶	common/security_vulnerability CWE-89 / CWE-79
P0	REST/AJAX/設定画面の権限・CSRF Nonce・ABSPATH 全面実装	全4 PHPファイル	未認証PII漏洩・権限昇格・CSRFデータ破壊を即時遮断	php/frameworks/wordpress common/security
P0	SEC-03/04/05 全脆弱性の一括封じ込めとテーブル接頭辞修正	全4 PHPファイル	無認証公開・インジェクション・直接ファイル実行を一括解消	common/security_vulnerability php/frameworks/wordpress
P0	ハードコード Webhook 秘密鍵の除去・失効と外部注入化	includes/ rest-endpoints.php	ソース流出時の Webhook 偽装・認証バイパスリスクを排除	common/security SEC-03
P1	wp_safe_remote_get(timeout=3s) 移行と PSR-3 構造化ログ整備	admin/settings-page.php rest-endpoints.php	外部遅延時のPHP-FPM枯渇防止と print_r JSON破壊の解消	common/production_readiness common/twelve_factor
P1	Composer + PSR-4 導入と Requires PHP: 8.2+ への刷新	member-analytics-portal.php	composer.lock による決定論的管理と EOL PHP 7.4 脱却	php/dependencies php/modern_stack
P1	PHPUnit + WP_UnitTestCase / WP_Ajax 自動テストスイート導入	全4 PHPファイル (tests/ 新設)	401/403認可拒否・Nonce検証・SQL安全性のCI自動回帰防止	php/testing
P1	オフライン実行可能な PHPUnit + PCOV カバレッジ検証構築	member-analytics-portal.php	RV-02 オフラインテスト合格と RV-03 実測網羅率 80%+ 達成	common/runtime_verification php/testing
P1	PSR-4 Repository 分離・strict_types 型宣言・wp_enqueue_script 化	全4 PHPファイル	型安全率・PHPDoc率 0%->100% と <script> 直書きの解消	php/typing php/code_quality
P2	README 仕様拡充と LICENSE / SECURITY / .gitignore / CI 整備	README.md	オンボーディング改善とサプライチェーン監査基準適合	common/documentation common/supply_chain

優先度順 改善アクション計画トレーサビリティマトリクス (P0 -> P1 -> P2 全10施策)

4. 評価指標とルール適合マトリクス

本章では、`php_wp_plugin_messy` ワークフローで実行した7つの評価ステップのスコア比較（弱点順）、全17エンジニアリングルールの適合判定マトリクス（`FAIL -> WARN -> NOT_APPLICABLE`）、および重要度順に正規化された全21件の指摘事項（`CRITICAL -> HIGH -> MEDIUM -> LOW`）を一覧化する。



実行監査ステップ別スコア比較 (弱点順: Weakest -> Strongest)

ルール適合マトリクス（全17ルール：`FAIL: 14` / `WARN: 2` / `PASS: 0` / `N/A: 1`）

評価対象の全17ルールは最悪ステータス優先で集約されており、`php/frameworks/wordpress` を含む **14ルールが** `FAIL`、2ルール（`common/git_workflow`・`common/maintainability`）が `WARN`、WordPress プラグインのため対象外となる `php/frameworks/laravel` が `NOT_APPLICABLE` となり、`PASS` 達成ルールは `0` 件であった。

判定	ルールID	代表評価ステップ	決定的エビデンス要約
FAIL	common/architecture	php_ast_audit	0クラス・7グローバル関数でHTTP入力/wpdb/外部通信/HTMLが混在
FAIL	common/documentation	repo_structure_audit	README.md が実質2行のみで前提・REST/AJAX仕様・テスト手順不在
FAIL	common/production_readiness	service_readiness_audit	settings-page.php:21 file_get_contents Timeout未設定・@抑制
FAIL	common/runtime_verification	container_runtime_audit	php -lと5フック登録は合格するが PHPUnit 0件・実測網羅率未計測
FAIL	common/security	repo_structure_audit	whsec直書き・REST '__return_true'・'read'権限・生SQL文字列結合
FAIL	common/security_vulnerability	php_ast_audit	CWE-89 SQLi 3件・CWE-79 XSS 4箇所・無認証API境界4件・OS層CVE
FAIL	common/supply_chain	repo_structure_audit	composer.json/lock・LICENSE・SECURITY.md・Dependabot・CI全欠落
FAIL	common/twelve_factor	service_readiness_audit	秘密鍵/外部URL/wp_map_events直書き・print_r標準出力デバッグ
FAIL	php/code_quality	php_ast_audit	PSR-4不在・手動require_once 3件・GET/POST直参照10件・PHPDoc 0%
FAIL	php/dependencies	repo_structure_audit	raw_php構成・Requires PHP: 7.4 (EOL) 指定・PSR-4オートロード未導入
FAIL	php/frameworks/wordpress	php_ast_audit	WP規約違反16件: ABSPATH欠落・__return_true・nopriv・Nonce不在
FAIL	php/modern_stack	repo_structure_audit	EOL PHP 7.4・file_get_contents無制限通信・print_r/echo json
FAIL	php/testing	php_test_audit	テストファイル0件・テスト関数0件・WP_UnitTestCase/PHPUnit不在
FAIL	php/typing	php_ast_audit	strict_types=1 0/4ファイル・引数/戻り値型宣言 0% (型問題11件)
WARN	common/git_workflow	repo_structure_audit	秘密ファイルコミットは0件だが .gitignore 不在でvendor等混入リスク
WARN	common/maintainability	php_ast_audit	関数自体は短いが @ エラー抑制・wp_接頭辞直書き・13箇所グローバル結合
N/A	php/frameworks/laravel	php_ast_audit	detected_frameworks=['wordpress'] のため Laravel 規約は対象外

エンジニアリングルール適合マトリクス (FAIL -> WARN -> N/A 全17ルール)

重要度順 指摘事項一覧（全21件：`CRITICAL: 7` / `HIGH: 7` / `MEDIUM: 6` / `LOW: 1`）

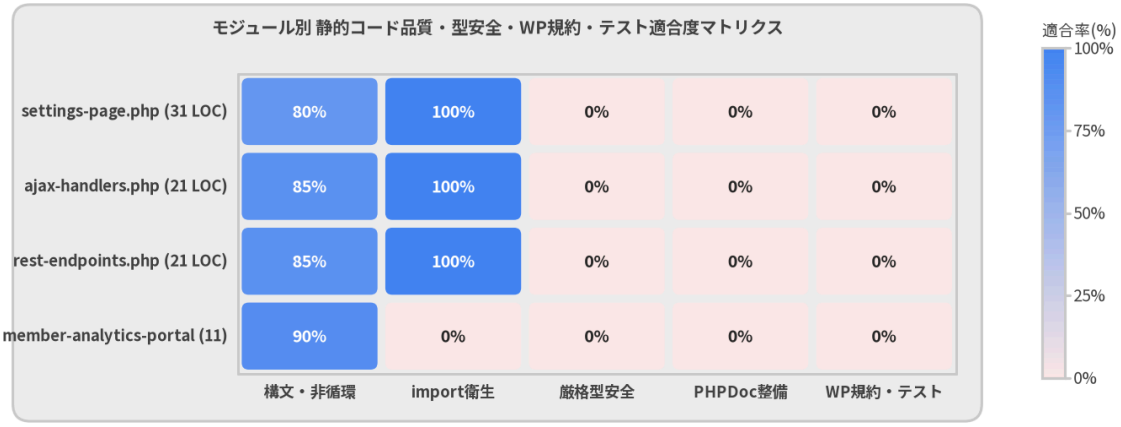
各ステップから抽出・重複排除された全21件の指摘事項を `CRITICAL -> HIGH -> MEDIUM -> LOW` の確定順序で以下に示す。

重要度	ルールID	該当箇所	検出事象・決定的根拠	推奨修正策
CRIT	common/security	rest-endpoints:3	map_webhook_secret ハードコード直書き	鍵失効・wp-config/getenv注入・hash_equals
CRIT	common/security_vulnerability	ajax-handlers:3	wp_ajax_nopriv 公開 + POST['term'] SQLi	nopriv廃止・Nonce/権限・wpdb->prepare
CRIT	common/security_vulnerability	ajax-handlers:10	SEC-04/05: 未認証 admin-ajax LIKE SQL結合	current_user_can + esc_like + prepare
CRIT	common/security_vulnerability	rest-endpoints:7	SEC-04/05: /export __return_true + SQLi	list_users権限・args検証・prepare適用
CRIT	php/frameworks/wordpress	settings-page:10	cap='read'登録 + Nonce不在のオプション更新	manage_options化 + check_admin_referer
CRIT	php/frameworks/wordpress	ajax-handlers:17	purge_logs Nonce/権限欠落 + DELETE SQLi/XSS	check_ajax_referer + prefix + prepare
CRIT	php/frameworks/wordpress	rest-endpoints:10	REST __return_true で全会員メールPII露出	permission_callback厳格化・print_r削除
HIGH	common/production_readiness	settings-page:21	file_get_contents Timeout未設定・@抑制	wp_safe_remote_get(timeout=3) + 非同期化
HIGH	php/dependencies	portal.php:7	Requires PHP: 7.4 (EOL)・composer/PSR-4不在	PHP ^8.2 + composer.json/lock + PSR-4化
HIGH	common/security_vulnerability	settings-page:26	GET['tab'] の <h1> / input value 未エスケープ	sanitize_key + esc_html / esc_attr 徹底
HIGH	php/frameworks/wordpress	portal.php:1	全4ファイルで先頭 ABSPATH 直接実行ガード欠落	全PHP先頭に if(!defined('ABSPATH')) exit
HIGH	php/frameworks/wordpress	portal.php:20	ショートコード GET['badge_label'] 反射XSS	shortcode_atts + sanitize + esc_html
HIGH	common/runtime_verification	portal.php:15	5フック登録合格に対し単体テスト0・網羅率未計測	PHPUnit + PCOV でオフライン網羅率80%+
HIGH	php/testing	portal.php:1	WP_UnitTestCase/PHPUnit 自動テスト完全不在	認可401/403・Nonce・SQL境界テスト追加
MED	php/code_quality	settings-page:27	<script>タグ直書き・13箇所グローバル状態結合	wp_enqueue_script化・Repository分離
MED	common/twelve_factor	rest-endpoints:21	?debug=1 で print_r(rows) 生データ標準出力	print_r削除・PSR-3 構造化JSONログ導入
MED	common/supply_chain	README.md:1	LICENSE / SECURITY.md / Dependabot / ignore不在	GPLv2+ LICENSE・SECURITY・CI整備
MED	common/supply_chain	portal.php:1	CIワークフロー不在で静的/動的検査が未自動化	最小権限 GitHub Actions CI の追加
MED	common/documentation	README.md:1	2行のみで前提・REST/AJAX仕様・テスト手順欠落	Standard Readme 準拠で仕様・手順明記
MED	php/typing	rest-endpoints:14	strict_types=1 0件・全7関数の型宣言欠落(0%)	declare(strict_types=1) + ネイティブ型付与
LOW	common/security_vulnerability	portal.php:7	composer.lock不在・Debian OS層 285件高深刻CVE	composerロック化・最小PHP 8.3+ベース採用

重要度順 検出指摘一覧 (CRITICAL -> HIGH -> MEDIUM -> LOW 全21件)

5. コード品質・型安全性・ランタイム検証詳細

本章では、PHP AST 静的解析によって抽出された4モジュールのコード品質・`declare(strict_types=1)` 型安全性・WordPress 規約適合度と、隔離 Docker コンテナ（PHP 8.2）上で実行された `php -l` 構文検証および WordPress フック登録スモークテスト（RV-01〜RV-04）の結果を詳細に分析する。



モジュール × 品質ディメンション適合度マトリクス (0 - 100%)

静的コード品質・型安全性・WordPress 実装規約の詳細評価

- **EOL ランタイム指定と型安全カバレッジ** `0.0%`：`member-analytics-portal.php:7` はセキュリティサポート終了済みの `Requires PHP: 7.4` を宣言している。全4ファイルで `declare(strict_types=1);` が欠落し、全7関数（`map_render_badge_shortcode`・`map_register_rest_routes`・`map_export_all_members`・`map_ajax_search_members`・`map_ajax_purge_logs`・`map_register_admin_menu`・`map_render_settings_page`）に戻り値型がなく、`$request`（`WP_REST_Request`）や `$atts` の引数型・PHPDoc（`0.0%`）も一切付与されていない。

- **グローバル結合・アセット直書き・ブロッキング外部通信**: 84 LOC 中に `global $wpdb` 結合が `3` 箇所、`$_GET` / `$_POST` の未サニタイズ直参照が `10` 箇所（計 `13` 箇所）存在する。`admin/settings-page.php:27` は `wp_enqueue_script()` を使わず `<script src="/wp-content/plugins/member-analytics-portal/assets/admin-chart.js"></script>` を直書きしており、サブディレクトリ型 WordPress 環境で 404 となる。また同 `21` 行目の `@file_get_contents('https://telemetry.example.com/ping?site=...')` はタイムアウト指定を欠き（デフォルト 60 秒ブロック）、`@` 演算子で通信失敗を無言で握り潰す危険なイディオムとなっている。さらに `includes/rest-endpoints.php:21` では `?debug=1` 時に `print_r($rows)` が JSON 出力を破壊する。

コンテナ動的検証（PHP 8.2 `php -l`・WordPress 5フック登録スモーク vs テスト不在）

隔離 Docker コンテナ（`php/raw.dockerfile`, `PHP_VERSION=8.2`）を用いた動的検証では、**構文検証と WordPress コアスタブによるプラグインロード自体は正常通過**した一方、**PHPUnit テスト 0 件によりコールバック内部の動的検証と実測カバレッジ計測が完全に欠落**している実態が明確化された。

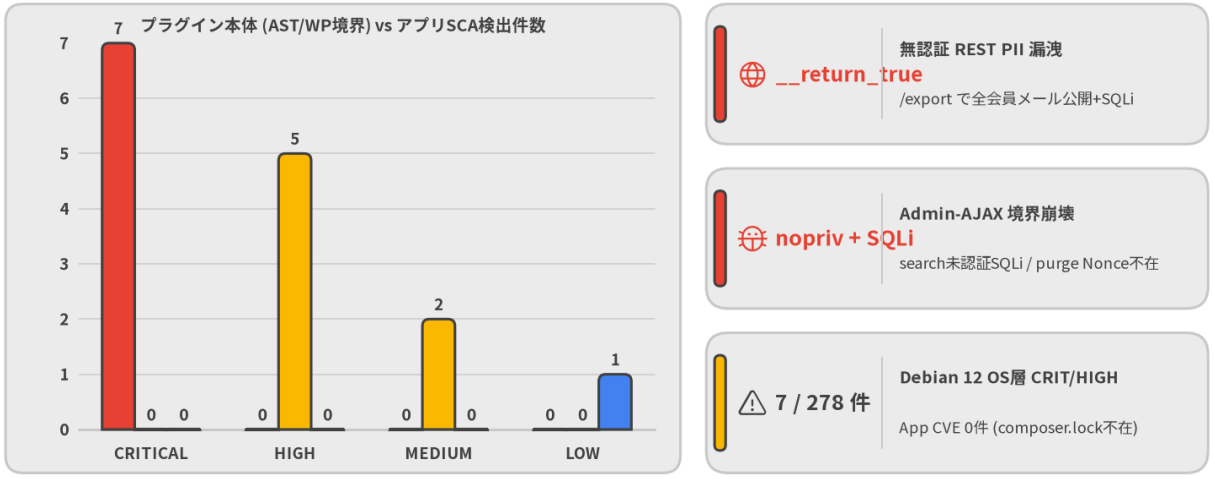


検証ゲート	ステータス	所要時間	決定的実行ログ・エビデンスと評価
RV-01: PHP 8.2 コンテナビルド	PASSED (exit 0)	60.24s	php/raw.dockerfile (PHP 8.2 / Debian 12.15) のクリーンビルドに成功。ただしリポジトリ自体には composer.json / lockfile が存在しない
RV-02a: php -l 構文ゲート	PASSED (exit 0)	0.80s	--network=none 環境で全4つの .php ファイルに対する php -l 構文検査が通過。PHP 8.2 での致命的な構文非互換はないことを確認
RV-02b: オフライン PHPUnit 実行	FAIL (不在) total_tests=0	0.00s	'PHP syntax gate (php -l) passed; no PHPUnit tests executed.' tests/ ディレクトリおよび *Test.php が0件のためコールバック未検証
RV-03: PCOV 実測カバレッジ	UNMEASURED (measured=false)	0.00s	line_coverage_pct=null, branch_coverage_pct=null。7関数すべての SQL生成・認可・エスケープ経路に対する実測網羅率が 0% 状態
RV-04: WP フック スモーク検証	PASSED (5 hooks)	0.59s	ABSPATH・add_action・add_shortcode スタブ下で member-analytics-portal.php が正常ロードされ、5フック（REST/Shortcode/AJAX×2/Admin）登録を確認

コンテナ動的検証KPIと RV-01〜RV-04 ゲート別実行結果内訳

6. セキュリティ・脆弱性・サプライチェーン分析

本章では、AST セキュリティ解析による WordPress 認可境界破綻（`SEC-05`）・CWE 危険コードシンク（`SEC-04`）・ハードコード秘密鍵（`SEC-03`）と、Trivy (`0.69.3`) によるファイルシステムおよび解決済みコンテナイメージ（Debian 12.15）の脆弱性スキャン結果（`SEC-01` / `SEC-02`）を統合して評価する。



深刻度別 セキュリティ問題内訳 (WordPress認可/CWEシンク vs コンテナOS層CVE)

セキュリティゲート（SEC-01〜SEC-05）および WordPress 固有脆弱性の詳細

- **SEC-05（API 認可境界・CSRF・権限昇格）**:
  1. **無認証 REST 会員エクスポート (`includes/rest-endpoints.php:7-12`)**: `GET /wp-json/member-analytics/v1/export` が `'permission_callback' => '__return_true'` で公開されており、未認証の外部攻撃者が `{$wpdb->users}` の `ID, user_login, user_email` を一括取得できる。

2. **未認証 Admin-AJAX 検索 (`includes/ajax-handlers.php:3,6-15`)**: `wp_ajax_nopriv_map_search_members` が未認証リクエストを受け付け、会員メールアドレスを返却する。
3. **Admin-AJAX ログ削除の Nonce/権限欠落 (`includes/ajax-handlers.php:4,17-24`)**: `wp_ajax_map_purge_logs` に `check_ajax_referer()` と `current_user_can()` がなく、購読者アカウントや CSRF 罠ページから `wp_map_events` を削除できる。
4. **購読者権限での設定画面登録と CSRF (`admin/settings-page.php:7-22`)**: `add_options_page()` の第3引数に `'read'`（購読者権限）を指定しており、かつ `check_admin_referer()` なしで `update_option('map_telemetry_url', ...)` と外部 HTTP 送信を実行する。
5. **全4ファイルの `ABSPATH` ガード不在**: 直接 HTTP リクエストに対する防御が存在しない。

- **SEC-04（CWE-89 SQLi 3箇所 & CWE-79 XSS 4箇所）**：`$wpdb->prepare()` を1箇所も使用しておらず、`includes/rest-endpoints.php:18-19`（`LIKE '%$role_filter%'`）、`includes/ajax-handlers.php:10-12`（`LIKE '%$term%'`）、`includes/ajax-handlers.php:21`（`DELETE ... WHERE category = '$category'`）の3クエリすべてが直接 SQL インジェクション可能である。また `admin/settings-page.php:26,29`（`$_GET['tab']`）、`includes/ajax-handlers.php:22`（`$_POST['category']`）、`member-analytics-portal.php:20-21`（`$_GET['badge_label']`）の計4箇所で未エスケープ出力による反射型 XSS が成立する。
- **SEC-03（ハードコード秘密鍵）& SEC-01（Trivy スキャン）**：`includes/rest-endpoints.php:3` に `$map_webhook_secret = 'whsec_[REDACTED]'` が平文でハードコードされている。Trivy スキャンでは `composer.lock` 不在のため `app_vuln_counts=0` だが、検証用 Debian 12.15 ベースイメージ OS 層で `CRITICAL: 7`（`CVE-2026-6653` `libxml2` CVSS 9.8、`CVE-2023-45853` `zlib1g` CVSS 9.8、`CVE-2025-7458` `libsqlite3-0` CVSS 9.1 等）・`HIGH: 278`（`CVE-2026-111497` `libbrotli1` -> 修正版 `1.0.9-2+deb12u1` 等、総計 `2,910` 件）が検出された。

ゲート / 深刻度	識別子・CWE	対象エンドポイント / 箇所	攻撃面・脆弱性メカニズム	必須緩和・修正策
SEC-05 + 04 [CRITICAL]	Broken Auth + CWE-89 SQLi	GET /wp-json/member-analytics/v1/export	permission_callback='__return_true' で会員PII露出 + role_filter SQL結合	current_user_can('list_users') + wpdb->prepare + esc_like
SEC-05 + 04 [CRITICAL]	Unauth AJAX + CWE-89 SQLi	POST admin-ajax.php ?action=map_search_members	wp_ajax_nopriv 未認証公開 + POST['term'] 生SQL結合・PII返却	nopriv廃止・check_ajax_referer・権限検査・wpdb->prepare
SEC-05 + 04 [CRITICAL]	CSRF / SQLi + CWE-79 XSS	POST admin-ajax.php ?action=map_purge_logs	Nonce/権限検査欠落 + wp_map_events DELETE SQLi + POST['category'] XSS	check_ajax_referer + manage_options + wpdb->prefix + prepare
SEC-05 [CRITICAL]	Privilege Esc + CSRF + SSRF	admin/settings-page.php :10-21 (cap='read')	購読者権限で設定アクセス・Nonce不在で update_option と外部HTTP送信	capability='manage_options' + wp_nonce_field / check_admin_referer
SEC-03 [CRITICAL]	Hardcoded Key (OWASP ASVS)	includes/rest-endpoints .php:3	map_webhook_secret に署名秘密鍵 (whsec_[REDACTED]) を平文直書き	既存キー失効・ローテーション・wp-config定数 + hash_equals検証
SEC-04 [HIGH]	CWE-79 XSS (Admin & Public)	settings-page:26,29 / portal.php:20-21	GET['tab'] とショートコードの GET['badge_label'] 未エスケープ出力	sanitize_key / sanitize_text_field + esc_html / esc_attr 適用
SEC-05 [HIGH]	Direct Access (No ABSPATH)	全4 PHPファイル先頭 (portal/rest/ajax/admin)	if (!defined('ABSPATH')) exit; 不在で直接URL実行時にパス漏洩リスク	全4ファイル先頭に ABSPATH 定数ガードを必須配置
SEC-01 [LOW / OS]	Trivy 0.69.3 SCA & OS Scan	composer不在 / Debian 12.15 (image.tar)	App CVE 0件(lock不在) / OS層 CRIT 7・HIGH 278 (libxml2, zlib1g, brotli)	composer.lock 導入によるSCA有効化・最小PHP 8.3+ ベースイメージ更新

WordPressエンドポイント攻撃面・CWE危険シンク・秘密鍵・Trivyスキャン統合マトリクス